

4.3.7 Interrupt priority registers (NVIC_IPRx)

Address offset: 0x00- 0x0B

Reset value: 0x0000 0000

Required privilege: Privileged

The NVIC_IPR0-IPR80 registers provide an 8-bit priority field for each interrupt. These registers are byte-accessible. Each register holds four priority fields, that map to four elements in the CMSIS interrupt priority array IP[0] to IP[67], as shown in [Figure 19](#).

Figure 19. NVIC_IPRx register mapping

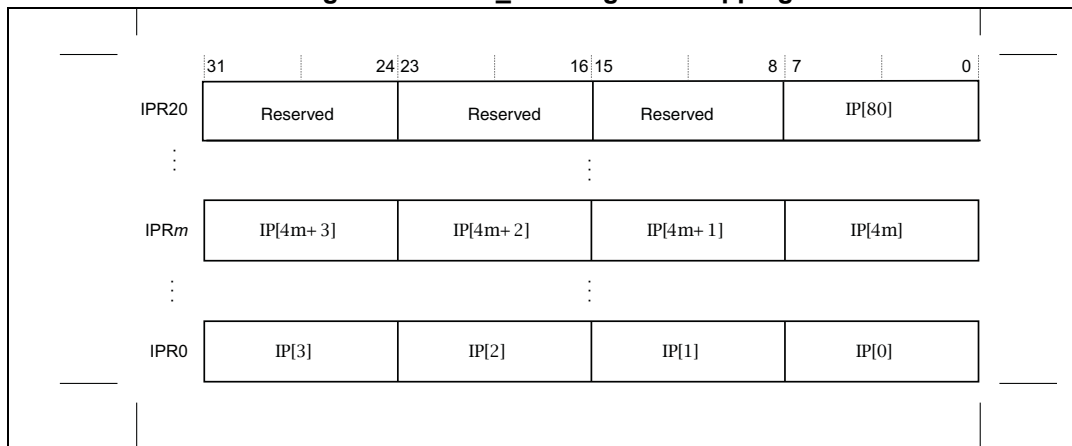


Table 47. IPR bit assignments

Bits	Name	Function
[31:24]	Priority, byte offset 3	Each priority field holds a priority value, 0-255. The lower the value, the greater the priority of the corresponding interrupt. The processor implements only bits[7:4] of each field, bits[3:0] read as zero and ignore writes.
[23:16]	Priority, byte offset 2	
[15:8]	Priority, byte offset 1	
[7:0]	Priority, byte offset 0	

See [Interrupt set-enable registers \(NVIC_ISERx\) on page 195](#) for more information about the interrupt priority array, that provides the software view of the interrupt priorities.

Find the IPR number and byte offset for interrupt N as follows:

- The corresponding IPR number, M , is given by $M = N \text{ DIV } 4$
- The byte offset of the required Priority field in this register is $N \text{ MOD } 4$, where:
 - byte offset 0 refers to register bits[7:0]
 - byte offset 1 refers to register bits[15:8]
 - byte offset 2 refers to register bits[23:16]
 - byte offset 3 refers to register bits[31:24].

4.3.8 Software trigger interrupt register (NVIC_STIR)

Address offset: 0xE00

Reset value: 0x0000 0000

Required privilege: When the USERSETMPEND bit in the SCR is set to 1, unprivileged software can access the STIR, see [Section 4.4.6: System control register \(SCR\)](#). Only privileged software can enable unprivileged access to the STIR.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							INTID[8:0]								
							w	w	w	w	w	w	w	w	w

Bits 31:9 Reserved, must be kept cleared.

Bits 8:0 **INTID** Software generated interrupt ID

Write to the STIR to generate a Software Generated Interrupt (SGI). The value to be written is the Interrupt ID of the required SGI, in the range 0-239. For example, a value of 0x03 specifies interrupt IRQ3.

4.3.9 Level-sensitive and pulse interrupts

STM32 interrupts are both level-sensitive and pulse-sensitive. Pulse interrupts are also described as edge-triggered interrupts.

A level-sensitive interrupt is held asserted until the peripheral deasserts the interrupt signal. Typically this happens because the ISR accesses the peripheral, causing it to clear the interrupt request. A pulse interrupt is an interrupt signal sampled synchronously on the rising edge of the processor clock. To ensure the NVIC detects the interrupt, the peripheral must assert the interrupt signal for at least one clock cycle, during which the NVIC detects the pulse and latches the interrupt.

When the processor enters the ISR, it automatically removes the pending state from the interrupt, see [Hardware and software control of interrupts](#). For a level-sensitive interrupt, if the signal is not deasserted before the processor returns from the ISR, the interrupt becomes pending again, and the processor must execute its ISR again. This means that the peripheral can hold the interrupt signal asserted until it no longer needs servicing.

Hardware and software control of interrupts

The Cortex-M4 latches all interrupts. A peripheral interrupt becomes pending for one of the following reasons:

- The NVIC detects that the interrupt signal is HIGH and the interrupt is not active
- The NVIC detects a rising edge on the interrupt signal
- Software writes to the corresponding interrupt set-pending register bit, see [Section 4.3.4: Interrupt set-pending registers \(NVIC_ISPRx\)](#), or to the STIR to make an SGI pending, see [Section 4.3.8: Software trigger interrupt register \(NVIC_STIR\)](#).

A pending interrupt remains pending until one of the following:

- The processor enters the ISR for the interrupt. This changes the state of the interrupt from pending to active. Then:
 - For a level-sensitive interrupt, when the processor returns from the ISR, the NVIC samples the interrupt signal. If the signal is asserted, the state of the interrupt changes to pending, which might cause the processor to immediately re-enter the ISR. Otherwise, the state of the interrupt changes to inactive.
 - For a pulse interrupt, the NVIC continues to monitor the interrupt signal, and if this is pulsed the state of the interrupt changes to pending and active. In this case, when the processor returns from the ISR the state of the interrupt changes to pending, which might cause the processor to immediately re-enter the ISR. If the interrupt signal is not pulsed while the processor is in the ISR, when the processor returns from the ISR the state of the interrupt changes to inactive.
- Software writes to the corresponding interrupt clear-pending register bit.

For a level-sensitive interrupt, if the interrupt signal is still asserted, the state of the interrupt does not change. Otherwise, the state of the interrupt changes to inactive.

For a pulse interrupt, state of the interrupt changes to:

 - Inactive, if the state was pending
 - Active, if the state was active and pending.

4.3.10 NVIC design hints and tips

Ensure software uses correctly aligned register accesses. The processor does not support unaligned accesses to NVIC registers. See the individual register descriptions for the supported access sizes.

An interrupt can enter pending state even it is disabled. Disabling an interrupt only prevents the processor from taking that interrupt.

Before programming VTOR to relocate the vector table, ensure the vector table entries of the new vector table are setup for fault handlers, NMI and all enabled exception like interrupts. For more information see [Section 4.4.4: Vector table offset register \(VTOR\) on page 212](#).

NVIC programming hints

Software uses the CPSIE I and CPSID I instructions to enable and disable interrupts. The CMSIS provides the following intrinsic functions for these instructions:

```
void __disable_irq(void) // Disable Interrupts
void __enable_irq(void) // Enable Interrupts
```

In addition, the CMSIS provides a number of functions for NVIC control, including:

Table 48. CMSIS functions for NVIC control

CMSIS interrupt control function	Description
void NVIC_SetPriorityGrouping(uint32_t priority_grouping)	Set the priority grouping
void NVIC_EnableIRQ(IRQn_t IRQn)	Enable IRQn
void NVIC_DisableIRQ(IRQn_t IRQn)	Disable IRQn
uint32_t NVIC_GetPendingIRQ (IRQn_t IRQn)	Return true (IRQ-Number) if IRQn is pending
void NVIC_SetPendingIRQ (IRQn_t IRQn)	Set IRQn pending
void NVIC_ClearPendingIRQ (IRQn_t IRQn)	Clear IRQn pending status
uint32_t NVIC_GetActive (IRQn_t IRQn)	Return the IRQ number of the active interrupt
void NVIC_SetPriority (IRQn_t IRQn, uint32_t priority)	Set priority for IRQn
uint32_t NVIC_GetPriority (IRQn_t IRQn)	Read priority of IRQn
void NVIC_SystemReset (void)	Reset the system

The input parameter IRQn is the IRQ number, see [Table 17: Properties of the different exception types on page 37](#). For more information about these functions see the CMSIS documentation.

This table shows the NVIC register map and reset values. The base address of the main NVIC register block is 0xE00E100. The NVIC_STIR register is located in a separate block at 0xE00EF00.

[illegible]

Table 49. NVIC register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x300	NVIC_IPR0	IP[3]								IP[2]								IP[1]								IP[0]							
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
:	:	:																															
0x320	NVIC_IPR20	Reserved																								IP[80]							
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
SCB registers																																	
Reserved																																	
0xE00	NVIC_STIR	Reserved																								INTID[8:0]							
	Reset Value																									0	0	0	0	0	0	0	0

4.4 System control block (SCB)

The *System control block* (SCB) provides system implementation information, and system control. This includes configuration, control, and reporting of the system exceptions.

Table 50. Summary of the system control block registers

Address	Name	Type	Required privilege	Reset value	Description
0xE000E008	ACTLR	RW	Privileged	0x00000000	Table 4.4.1: Auxiliary control register (ACTLR) on page 207
0xE000ED00	CPUID	RO	Privileged	0x410FC241	Table 4.4.2: CPUID base register (CPUID) on page 208
0xE000ED04	ICSR	RW ⁽¹⁾	Privileged	0x00000000	Table 4.4.3: Interrupt control and state register (ICSR) on page 210
0xE000ED08	VTOR	RW	Privileged	0x00000000	Table 4.4.4: Vector table offset register (VTOR) on page 212
0xE000ED0C	AIRCR	RW ⁽¹⁾	Privileged	0xFA050000	Table 4.4.5: Application interrupt and reset control register (AIRCR) on page 212
0xE000ED10	SCR	RW	Privileged	0x00000000	Table 4.4.6: System control register (SCR) on page 214
0xE000ED14	CCR	RW	Privileged	0x00000200	Table 4.4.7: Configuration and control register (CCR) on page 215
0xE000ED18	SHPR1	RW	Privileged	0x00000000	Table 4.4.8: System handler priority registers (SHPRx) on page 217
0xE000ED1C	SHPR2	RW	Privileged	0x00000000	
0xE000ED20	SHPR3	RW	Privileged	0x00000000	
0xE000ED24	SHCRS	RW	Privileged	0x00000000	Table 4.4.9: System handler control and state register (SHCSR) on page 219
0xE000ED28	CFSR	RW	Privileged	0x00000000	Table 4.4.10: Configurable fault status register (CFSR; UFSR+BFSR+MMFSR) on page 221
0xE000ED28	MMSR ⁽²⁾	RW	Privileged	0x00	MemManage Fault Status Register Table 4.4.10 on page 221
0xE000ED29	BFSR ⁽²⁾	RW	Privileged	0x00	BusFault Status Register Table 4.4.10 on page 221
0xE000ED2A	UFSR ⁽²⁾	RW	Privileged	0x0000	UsageFault Status Register Table 4.4.10 on page 221
0xE000ED2C	HFSR	RW	Privileged	0x00000000	Table 4.4.14: Hard fault status register (HFSR) on page 225
0xE000ED34	MMAR	RW	Privileged	Unknown	Table 4.4.15: Memory management fault address register (MMFAR) on page 226
0xE000ED38	BFAR	RW	Privileged	Unknown	Table 4.4.16: Bus fault address register (BFAR) on page 226
0xE000ED3C	AFSR	RW	Privileged	0x00000000	Table 4.4.17: Auxiliary fault status register (AFSR) on page 227

1. See the register description for more information.

2. A subregister of the CFSR

4.4.1 Auxiliary control register (ACTLR)

Address offset: 0x00 (base address = 0xE000 E008)

Reset value: 0x0000 0000

Required privilege: Privileged

By default this register is set to provide optimum performance from the Cortex-M4 processor, and does not normally require modification. The ACTLR register provides disable bits for the following processor functions:

- IT folding
- write buffer use for accesses to the default memory map
- interruption of multi-cycle instructions.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						DISOO FP	DISFP CA						DISFOL D	DISDE FWBUF	DISMC YCINT
						rw	rw						rw	rw	rw

Bits 31:10 Reserved

Bit 9 DISOOF

Disables floating point instructions completing out of order with respect to integer instructions.

Bit 8 DISFPCA

Disables automatic update of CONTROL.FPCA.

The value of this bit should be written as zero or preserved (SBZP).

Bit 7:3 Reserved

Bit 2 DISFOLD

Disables folding of IT instructions:

- 0: Enables IT instructions folding.
- 1: Disables IT instructions folding.

In some situations, the processor can start executing the first instruction in an IT block while it is still executing the IT instruction. This behavior is called IT folding, and improves performance. However, IT folding can cause jitter in looping. If a task must avoid jitter, set the DISFOLD bit to 1 before executing the task, to disable IT folding.

Bit 1 DISDEFWBUF

This bit only affects write buffers implemented in the Cortex-M4 processor.

Disables write buffer use during default memory map accesses: This causes all BusFaults to be precise BusFaults but decreases performance because any store to memory must complete before the processor can execute the next instruction.

- 0: Enable write buffer use: stores to memory is completed before next instruction.
- 1: Disable write buffer use.

Bit 0 DISMCYCINT

Disables interrupt of multi-cycle instructions. When set to 1, disables interruption of load multiple and store multiple instructions. This increases the interrupt latency of the processor because any LDM or STM must complete before the processor can stack the current state and enter the interrupt handler.

- 0: Enable interruption latency of the processor (load/store and multiply/divide operations).
- 1: Disable interruptions latency.

4.4.2 CPUID base register (CPUID)

Address offset: 0x00

Reset value: 0x410F C241

Required privilege: Privileged

The CPUID register contains the processor part number, version, and implementation information.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Implementer								Variant				Constant			
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PartNo												Revision			
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:24 **Implementer**: Implementer code

0x41: ARM

Bits 23:20 **Variant**: Variant number

The r value in the *rnpr* product revision identifier

0x0: revision 0

Bits 19:16 **Constant:** Reads as 0xF

Bits 15:4 **PartNo:** Part number of the processor
0xC24: = Cortex-M4

Bits 3:0 **Revision:** Revision number

The p value in the *rnpr* product revision identifier, indicates patch release.
0x1: = patch 1

4.4.3 Interrupt control and state register (ICSR)

Address offset: 0x04

Reset value: 0x0000 0000

Required privilege: Privileged

The ICSR:

- Provides:
 - A set-pending bit for the *Non-Maskable Interrupt* (NMI) exception
 - Set-pending and clear-pending bits for the PendSV and SysTick exceptions
- Indicates:
 - The exception number of the exception being processed
 - Whether there are preempted active exceptions
 - The exception number of the highest priority pending exception
 - Whether any interrupts are pending.

Caution: When you write to the ICSR, the effect is unpredictable if you:

- Write 1 to the PENDSVSET bit and write 1 to the PENDSVCLR bit
- Write 1 to the PENDSTSET bit and write 1 to the PENDSTCLR bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
NMIPENDSET	Reserved			PENDSVSET	PENDSVCLR	PENDSTSET	PENDSTCLR	Reserved			ISR_PENDING	Reserved			VECTPENDING[6:4]
rw				rw	w	rw	w				r				r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VECTPENDING[3:0]				RETOBASE	Reserved			VECTACTIVE[8:0]							
r	r	r	r	r				rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **NMIPENDSET**: NMI set-pending bit.

Write:

0: No effect

1: Change NMI exception state to pending.

Read:

0: NMI exception is not pending

1: NMI exception is pending

Because NMI is the highest-priority exception, normally the processor enters the NMI exception handler as soon as it registers a write of 1 to this bit, and entering the handler clears this bit to 0. A read of this bit by the NMI exception handler returns 1 only if the NMI signal is reasserted while the processor is executing that handler.

Bits 30:29 Reserved

- Bit 28 **PENDSVSET**: PendSV set-pending bit.
 Write:
 0: No effect
 1: Change PendSV exception state to pending.
 Read:
 0: PendSV exception is not pending
 1: PendSV exception is pending
 Writing 1 to this bit is the only way to set the PendSV exception state to pending.
- Bit 27 **PENDSVCLR**: PendSV clear-pending bit. This bit is write-only. On a read, value is unknown.
 0: No effect
 1: Removes the pending state from the PendSV exception.
- Bit 26 **PENDSTSET**: SysTick exception set-pending bit.
Write:
 0: No effect
 1: Change SysTick exception state to pending
Read:
 0: SysTick exception is not pending
 1: SysTick exception is pending
- Bit 25 **PENDSTCLR**: SysTick exception clear-pending bit. Write-only. On a read, value is unknown.
 0: No effect
 1: Removes the pending state from the SysTick exception.
- Bit 24 Reserved, must be kept cleared.
- Bit 23 This bit is reserved for Debug use and reads-as-zero when the processor is not in Debug.
- Bit 22 **ISR_PENDING**: Interrupt pending flag, excluding NMI and Faults.
 0: Interrupt not pending
 1: Interrupt pending
- Bits 21:19 Reserved, must be kept cleared.
- Bits 18:12 **VECTPENDING**: Pending vector. Indicates the exception number of the highest priority pending enabled exception.
 0: No pending exceptions
 Other values: The exception number of the highest priority pending enabled exception.
 The value indicated by this field includes the effect of the BASEPRI and FAULTMASK registers, but not any effect of the PRIMASK register.
- Bit 11 **RETTOBASE**: Return to base level. Indicates whether there are preempted active exceptions:
 0: There are preempted active exceptions to execute
 1: There are no active exceptions, or the currently-executing exception is the only active exception.
- Bits 10:9 Reserved
- Bits 8:0 **VECTACTIVE** Active vector. Contains the active exception number:
 0: Thread mode
 Other values: The exception number⁽¹⁾ of the currently active exception.
Note: Subtract 16 from this value to obtain CMSIS IRQ number required to index into the Interrupt Clear-Enable, Set-Enable, Clear-Pending, Set-Pending, or Priority Registers, see Table 6 on page 21.

1. This is the same value as IPSR bits[8:0], see [Interrupt program status register on page 21](#).

4.4.4 Vector table offset register (VTOR)

Address offset: 0x08

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		TBLOFF[29:16]													
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TBLOFF[15:9]							Reserved								
rw	rw	rw	rw	rw	rw	rw									

Bits 31:30 Reserved, must be kept cleared

Bits 29:9 **TBLOFF**: Vector table base offset field.

It contains bits [29:9] of the offset of the table base from memory address 0x00000000. When setting TBLOFF, you must align the offset to the number of exception entries in the vector table. The minimum alignment is 128 words. Table alignment requirements mean that bits[8:0] of the table offset are always zero.

Bit 29 determines whether the vector table is in the code or SRAM memory region.

0: Code

1: SRAM

Note: Bit 29 is sometimes called the TBLBASE bit.

Bits 8:0 Reserved, must be kept cleared

4.4.5 Application interrupt and reset control register (AIRCRR)

Address offset: 0x0C

Reset value: 0xFA05 0000

Required privilege: Privileged

The AIRCRR provides priority grouping control for the exception model, endian status for data accesses, and reset control of the system.

To write to this register, you must write 0x5FA to the VECTKEY field, otherwise the processor ignores the write.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
VECTKEYSTAT[15:0](read)/ VECTKEY[15:0](write)															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ENDIANESS	Reserved					PRIGROUP			Reserved				SYS RESET REQ	VECT CLR ACTIVE	VECT RESET
r						rw	rw	rw					w	w	w

Bits 31:16 **VECTKEYSTAT/ VECTKEY** Register key

Reads as 0xFA05

On writes, write 0x5FA to VECTKEY, otherwise the write is ignored.

Bit 15 **ENDIANESS** Data endianness bit

Reads as 0.

0: Little-endian

Bits 14:11 Reserved, must be kept cleared

Bits 10:8 **PRIGROUP**: Interrupt priority grouping field

This field determines the split of group priority from subpriority, see [Binary point on page 213](#).

Bits 7:3 Reserved, must be kept cleared

Bit 2 **SYSRESETREQ** System reset request

This is intended to force a large system reset of all major components except for debug.

This bit reads as 0.

0: No system reset request

1: Asserts a signal to the outer system that requests a reset.

Bit 1 **VECTCLRACTIVE**

Reserved for Debug use. This bit reads as 0. When writing to the register you must write 0 to this bit, otherwise behavior is unpredictable.

Bit 0 **VECTRESET**

Reserved for Debug use. This bit reads as 0. When writing to the register you must write 0 to this bit, otherwise behavior is unpredictable.

Binary point

The PRIGROUP field indicates the position of the binary point that splits the PRI_n fields in the Interrupt Priority Registers into separate *group priority* and *subpriority* fields. [Table 51](#) shows how the PRIGROUP value controls this split. If you implement fewer than 8 priority bits you might require more explanation here, and want to remove invalid rows from the table, and modify the entries in the number of columns.

Table 51. Priority grouping

PRIGROUP [2:0]	Interrupt priority level value, PRI _N [7:4]			Number of	
	Binary point ⁽¹⁾	Group priority bits	Subpriority bits	Group priorities	Sub priorities
0b011	0bxxxx	[7:4]	None	16	None
0b100	0bxxx.y	[7:5]	[4]	8	2
0b101	0bxx.yy	[7:6]	[5:4]	4	4
0b110	0bx.yyy	[7]	[6:4]	2	8
0b111	0b.yyyy	None	[7:4]	None	16

1. PRI_n[7:4] field showing the binary point. x denotes a group priority field bit, and y denotes a subpriority field bit.

Determining preemption of an exception uses only the group priority field, see [Section 2.3.6: Interrupt priority grouping on page 40](#).

4.4.6 System control register (SCR)

Address offset: 0x10

Reset value: 0x0000 0000

Required privilege: Privileged

The SCR controls features of entry to and exit from low power state.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved											SEVON PEND	Res.	SLEEP DEEP	SLEEP ON EXIT	Res.
											rw		rw	rw	

Bits 31:5 Reserved, must be kept cleared

Bit 4 **SEVONPEND** Send Event on Pending bit

When an event or interrupt enters pending state, the event signal wakes up the processor from WFE. If the processor is not waiting for an event, the event is registered and affects the next WFE.

The processor also wakes up on execution of an SEV instruction or an external event

0: Only enabled interrupts or events can wakeup the processor, disabled interrupts are excluded

1: Enabled events and all interrupts, including disabled interrupts, can wakeup the processor.

Bit 3 Reserved, must be kept cleared

Bit 2 **SLEEPDEEP**

Controls whether the processor uses sleep or deep sleep as its low power mode:

0: Sleep

1: Deep sleep.

Bit 1 **SLEEPONEXIT**

Configures sleep-on-exit when returning from Handler mode to Thread mode. Setting this bit to 1 enables an interrupt-driven application to avoid returning to an empty main application.

0: Do not sleep when returning to Thread mode.

1: Enter sleep, or deep sleep, on return from an interrupt service routine.

Bit 0 Reserved, must be kept cleared

4.4.7 Configuration and control register (CCR)

Address offset: 0x14

Reset value: 0x0000 0200

Required privilege: Privileged

The CCR controls entry to Thread mode and enables:

- The handlers for NMI, hard fault and faults escalated by FAULTMASK to ignore bus faults
- Trapping of divide by zero and unaligned accesses
- Access to the STIR by unprivileged software, see [Software trigger interrupt register \(NVIC_STIR\) on page 201](#).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						STK ALIGN	BFHF NMIGN	Reserved			DIV_0_ TRP	UN ALIGN_ TRP	Res.	USER SET MPEND	NON BASE THRD ENA
						rw	rw				rw	rw		rw	rw

Bits 31:10 Reserved, must be kept cleared

Bit 9 STKALIGN

Configures stack alignment on exception entry. On exception entry, the processor uses bit 9 of the stacked PSR to indicate the stack alignment. On return from the exception it uses this stacked bit to restore the correct stack alignment.

0: 4-byte aligned

1: 8-byte aligned

Bit 8 BFHFNMIGN

Enables handlers with priority -1 or -2 to ignore data bus faults caused by load and store instructions. This applies to the hard fault, NMI, and FAULTMASK escalated handlers. Set this bit to 1 only when the handler and its data are in absolutely safe memory. The normal use of this bit is to probe system devices and bridges to detect control path problems and fix them.

0: Data bus faults caused by load and store instructions cause a lock-up

1: Handlers running at priority -1 and -2 ignore data bus faults caused by load and store instructions.

Bits 7:5 Reserved, must be kept cleared

Bit 4 DIV_0_TRP

Enables faulting or halting when the processor executes an SDIV or UDIV instruction with a divisor of 0:

0: Do not trap divide by 0

1: Trap divide by 0.

When this bit is set to 0, a divide by zero returns a quotient of 0.

Bit 3 UNALIGN_TRP

Enables unaligned access traps:

- 0: Do not trap unaligned halfword and word accesses
- 1: Trap unaligned halfword and word accesses.

If this bit is set to 1, an unaligned access generates a usage fault.

Unaligned LDM, STM, LDRD, and STRD instructions always fault irrespective of whether UNALIGN_TRP is set to 1.

Bit 2 Reserved, must be kept cleared

Bit 1 USERSETMPEND

Enables unprivileged software access to the STIR, see [Software trigger interrupt register \(NVIC_STIR\) on page 201](#):

- 0: Disable
- 1: Enable.

Bit 0 NONBASETHRDENA

Configures how the processor enters Thread mode.

- 0: Processor can enter Thread mode only when no exception is active.
- 1: Processor can enter Thread mode from any level under the control of an EXC_RETURN value, see [Exception return on page 43](#).

4.4.8 System handler priority registers (SHPRx)

The SHPR1-SHPR3 registers set the priority level, 0 to 255 of the exception handlers that have configurable priority.

SHPR1-SHPR3 are byte accessible.

The system fault handlers and the priority field and register for each handler are:

Table 52. System fault handler priority fields

Handler	Field	Register description
Memory management fault	PRI_4	<i>System handler priority register 1 (SHPR1)</i>
Bus fault	PRI_5	
Usage fault	PRI_6	
SVCall	PRI_11	<i>System handler priority register 2 (SHPR2) on page 217</i>
PendSV	PRI_14	<i>System handler priority register 3 (SHPR3) on page 218</i>
SysTick	PRI_15	

Each PRI_N field is 8 bits wide, but the processor implements only bits[7:3] of each field, and bits[3:0] read as zero and ignore writes (where M=4).

System handler priority register 1 (SHPR1)

Address offset: 0x18

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								PRI_6[7:4]				PRI_6[3:0]			
								rw	rw	rw	rw	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRI_5[7:4]				PRI_5[3:0]				PRI_4[7:4]				PRI_4[3:0]			
rw	rw	rw	rw	r	r	r	r	rw	rw	rw	rw	r	r	r	r

Bits 31:24 Reserved, must be kept cleared

Bits 23:16 PRI_6: Priority of system handler 6, usage fault

Bits 15:8 PRI_5: Priority of system handler 5, bus fault

Bits 7:0 PRI_4: Priority of system handler 4, memory management fault

System handler priority register 2 (SHPR2)

Address offset: 0x1C

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRI_11[7:4]				PRI_11[3:0]				Reserved							
rw	rw	rw	rw	r	r	r	r								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

Bits 31:24 **PRI_11**: Priority of system handler 11, SVCall

Bits 23:0 Reserved, must be kept cleared

System handler priority register 3 (SHPR3)

Address: 0xE000 ED20

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRI_15[7:4]				PRI_15[3:0]				PRI_14[7:4]				PRI_14[3:0]			
rw	rw	rw	rw	r	r	r	r	rw	rw	rw	rw	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

Bits 31:24 **PRI_15**: Priority of system handler 15, SysTick exception

Bits 23:16 **PRI_14**: Priority of system handler 14, PendSV

Bits 15:0 Reserved, must be kept cleared

4.4.9 System handler control and state register (SHCSR)

Address offset: 0x24

Reset value: 0x0000 0000

Required privilege: Privileged

The SHCSR enables the system handlers, and indicates:

- The pending status of the bus fault, memory management fault, and SVC exceptions
- The active status of the system handlers.

If you disable a system handler and the corresponding fault occurs, the processor treats the fault as a hard fault.

You can write to this register to change the pending or active status of system exceptions. An OS kernel can write to the active bits to perform a context switch that changes the current exception type.

- Software that changes the value of an active bit in this register without correct adjustment to the stacked content can cause the processor to generate a fault exception. Ensure software that writes to this register retains and subsequently restores the current active status.
- After you have enabled the system handlers, if you have to change the value of a bit in this register you must use a read-modify-write procedure to ensure that you change only the required bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved													USG FAULT ENA	BUS FAULT ENA	MEM FAULT ENA
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SV CALL PEND ED	BUS FAULT PEND ED	MEM FAULT PEND ED	USG FAULT PEND ED	SYS TICK ACT	PEND SV ACT	Res.	MONIT OR ACT	SV CALL ACT	Reserved			USG FAULT ACT	Res.	BUS FAULT ACT	MEM FAULT ACT
rw	rw	rw	rw	rw	rw		rw	rw				rw		rw	rw

Bits 31:19 Reserved, must be kept cleared

Bit 18 **USGFAULTENA**: Usage fault enable bit, set to 1 to enable⁽¹⁾

Bit 17 **BUSFAULTENA**: Bus fault enable bit, set to 1 to enable⁽¹⁾

Bit 16 **MEMFAULTENA**: Memory management fault enable bit, set to 1 to enable⁽¹⁾

Bit 15 **SVCALLPENDE**: SVC call pending bit, reads as 1 if exception is pending⁽²⁾

Bit 14 **BUSFAULTPENDE**: Bus fault exception pending bit, reads as 1 if exception is pending⁽²⁾

Bit 13 **MEMFAULTPENDE**: Memory management fault exception pending bit, reads as 1 if exception is pending⁽²⁾

Bit 12 **USGFAULTPENDE**: Usage fault exception pending bit, reads as 1 if exception is pending⁽²⁾

Bit 11 **SYSTICKACT**: SysTick exception active bit, reads as 1 if exception is active⁽³⁾

Bit 10 **PENDSVACT**: PendSV exception active bit, reads as 1 if exception is active

Bit 9 Reserved, must be kept cleared

Bit 8 **MONITORACT**: Debug monitor active bit, reads as 1 if Debug monitor is active

Bit 7 **SVCALLACT**: SVC call active bit, reads as 1 if SVC call is active

Bits 6:4 Reserved, must be kept cleared

Bit 3 **USGFAULTACT**: Usage fault exception active bit, reads as 1 if exception is active

Bit 2 Reserved, must be kept cleared

Bit 1 **BUSFAULTACT**: Bus fault exception active bit, reads as 1 if exception is active

Bit 0 **MEMFAULTACT**: Memory management fault exception active bit, reads as 1 if exception is active

1. Enable bits, set to 1 to enable the exception, or set to 0 to disable the exception.
2. Pending bits, read as 1 if the exception is pending, or as 0 if it is not pending. You can write to these bits to change the pending status of the exceptions.
3. Active bits, read as 1 if the exception is active, or as 0 if it is not active. You can write to these bits to change the active status of the exceptions, but see the Caution in this section.

4.4.10 Configurable fault status register (CFSR; UFSR+BFSR+MMFSR)

Address offset: 0x28

Reset value: 0x0000 0000

Required privilege: Privileged

The following subsections describe the subregisters that make up the CFSR:

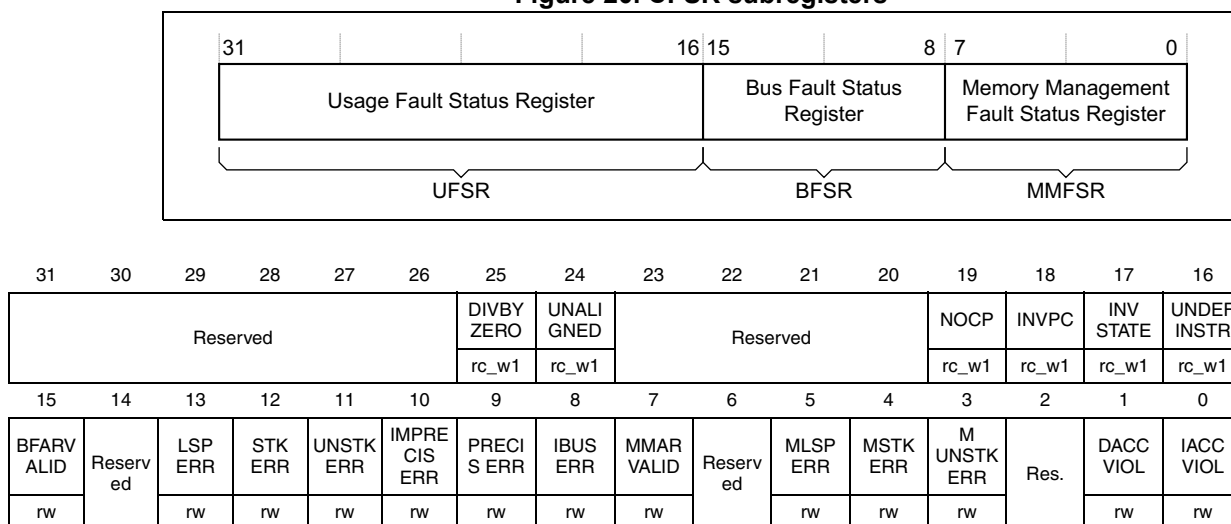
- [Usage fault status register \(UFSR\) on page 222](#)
- [Bus fault status register \(BFSR\) on page 223](#)
- [Memory management fault address register \(MMFSR\) on page 224](#)

The CFSR is byte accessible. You can access the CFSR or its subregisters as follows:

- Access the complete CFSR with a word access to 0xE000ED28
- Access the MMFSR with a byte access to 0xE000ED28
- Access the MMFSR and BFSR with a halfword access to 0xE000ED28
- Access the BFSR with a byte access to 0xE000ED29
- Access the UFSR with a halfword access to 0xE000ED2A.

The CFSR indicates the cause of a memory management fault, bus fault, or usage fault.

Figure 20. CFSR subregisters



Bits 31:16 **UFSR**: see [Usage fault status register \(UFSR\) on page 222](#)

Bits 15:8 **BFSR**: see [Bus fault status register \(BFSR\) on page 223](#)

Bits 7:0 **MMFSR**: see [Memory management fault address register \(MMFSR\) on page 224](#)

4.4.11 Usage fault status register (UFSR)

Bits 31:26 Reserved, must be kept cleared

- Bit 25 **DIVBYZERO**: Divide by zero usage fault. When the processor sets this bit to 1, the PC value stacked for the exception return points to the instruction that performed the divide by zero. Enable trapping of divide by zero by setting the DIV_0_TRP bit in the CCR to 1, see [Configuration and control register \(CCR\) on page 215](#).
- 0: No divide by zero fault, or divide by zero trapping not enabled
 - 1: The processor has executed an SDIV or UDIV instruction with a divisor of 0.

- Bit 24 **UNALIGNED**: Unaligned access usage fault. Enable trapping of unaligned accesses by setting the UNALIGN_TRP bit in the CCR to 1, see [Configuration and control register \(CCR\) on page 215](#).

Unaligned LDM, STM, LDRD, and STRD instructions always fault irrespective of the setting of UNALIGN_TRP.

- 0: No unaligned access fault, or unaligned access trapping not enabled
- 1: the processor has made an unaligned memory access.

Bits 23:20 Reserved, must be kept cleared

- Bit 19 **NOCP**: No coprocessor usage fault. The processor does not support coprocessor instructions:
- 0: No usage fault caused by attempting to access a coprocessor
 - 1: the processor has attempted to access a coprocessor.

- Bit 18 **INVPC**: Invalid PC load usage fault, caused by an invalid PC load by EXC_RETURN:
- When this bit is set to 1, the PC value stacked for the exception return points to the instruction that tried to perform the illegal load of the PC.
- 0: No invalid PC load usage fault
 - 1: The processor has attempted an illegal load of EXC_RETURN to the PC, as a result of an invalid context, or an invalid EXC_RETURN value.

- Bit 17 **INVSTATE**: Invalid state usage fault. When this bit is set to 1, the PC value stacked for the exception return points to the instruction that attempted the illegal use of the EPSR. This bit is not set to 1 if an undefined instruction uses the EPSR.
- 0: No invalid state usage fault
 - 1: The processor has attempted to execute an instruction that makes illegal use of the EPSR.

- Bit 16 **UNDEFINSTR**: Undefined instruction usage fault. When this bit is set to 1, the PC value stacked for the exception return points to the undefined instruction.
- An undefined instruction is an instruction that the processor cannot decode.
- 0: No undefined instruction usage fault
 - 1: The processor has attempted to execute an undefined instruction.

4.4.12 Bus fault status register (BFSR)

- Bit 15 **BFARVALID**: *Bus Fault Address Register* (BFAR) valid flag. The processor sets this bit to 1 after a bus fault where the address is known. Other faults can set this bit to 0, such as a memory management fault occurring later.
- If a bus fault occurs and is escalated to a hard fault because of priority, the hard fault handler must set this bit to 0. This prevents problems if returning to a stacked active bus fault handler whose BFAR value has been overwritten.
- 0: Value in BFAR is not a valid fault address
 - 1: BFAR holds a valid fault address.
- Bit 14 Reserved, must be kept cleared
- Bit 13 **LSPERR**: Bus fault on floating-point lazy state preservation.
- 0: No bus fault occurred during floating-point lazy state preservation.
 - 1: A bus fault occurred during floating-point lazy state preservation
- Bit 12 **STKERR**: Bus fault on stacking for exception entry. When the processor sets this bit to 1, the SP is still adjusted but the values in the context area on the stack might be incorrect. The processor does not write a fault address to the BFAR.
- 0: No stacking fault
 - 1: Stacking for an exception entry has caused one or more bus faults.
- Bit 11 **UNSTKERR**: Bus fault on unstacking for a return from exception. This fault is chained to the handler. This means that when the processor sets this bit to 1, the original return stack is still present. The processor does not adjust the SP from the failing return, does not performed a new save, and does not write a fault address to the BFAR.
- 0: No unstacking fault
 - 1: Unstack for an exception return has caused one or more bus faults.
- Bit 10 **IMPRECISERR**: Imprecise data bus error. When the processor sets this bit to 1, it does not write a fault address to the BFAR. This is an asynchronous fault. Therefore, if it is detected when the priority of the current process is higher than the bus fault priority, the bus fault becomes pending and becomes active only when the processor returns from all higher priority processes. If a precise fault occurs before the processor enters the handler for the imprecise bus fault, the handler detects both IMPRECISERR set to 1 and one of the precise fault status bits set to 1.
- 0: No imprecise data bus error
 - 1: A data bus error has occurred, but the return address in the stack frame is not related to the instruction that caused the error.
- Bit 9 **PRECISERR**: Precise data bus error. When the processor sets this bit is 1, it writes the faulting address to the BFAR.
- 0: No precise data bus error
 - 1: A data bus error has occurred, and the PC value stacked for the exception return points to the instruction that caused the fault.
- Bit 8 **IBUSERR**: Instruction bus error. The processor detects the instruction bus error on prefetching an instruction, but it sets the IBUSERR flag to 1 only if it attempts to issue the faulting instruction.
- When the processor sets this bit is 1, it does not write a fault address to the BFAR.
- 0: No instruction bus error
 - 1: Instruction bus error.

4.4.13 Memory management fault address register (MMFSR)

- Bit 7 **MMARVALID**: Memory Management Fault Address Register (MMAR) valid flag. If a memory management fault occurs and is escalated to a hard fault because of priority, the hard fault handler must set this bit to 0. This prevents problems on return to a stacked active memory management fault handler whose MMAR value has been overwritten.
- 0: Value in MMAR is not a valid fault address
 - 1: MMAR holds a valid fault address.
- Bit 6 Reserved, must be kept cleared
- Bit 5 **MLSPERR**:
- 0: No MemManage fault occurred during floating-point lazy state preservation
 - 1: A MemManage fault occurred during floating-point lazy state preservation
- Bit 4 **MSTKERR**: Memory manager fault on stacking for exception entry. When this bit is 1, the SP is still adjusted but the values in the context area on the stack might be incorrect. The processor has not written a fault address to the MMAR.
- 0: No stacking fault
 - 1: Stacking for an exception entry has caused one or more access violations.
- Bit 3 **MUNSTKERR**: Memory manager fault on unstacking for a return from exception. This fault is chained to the handler. This means that when this bit is 1, the original return stack is still present. The processor has not adjusted the SP from the failing return, and has not performed a new save. The processor has not written a fault address to the MMAR.
- 0: No unstacking fault
 - 1: Unstack for an exception return has caused one or more access violations.
- Bit 2 Reserved, must be kept cleared
- Bit 1 **DACCVIOL**: Data access violation flag. When this bit is 1, the PC value stacked for the exception return points to the faulting instruction. The processor has loaded the MMAR with the address of the attempted access.
- 0: No data access violation fault
 - 1: The processor attempted a load or store at a location that does not permit the operation.
- Bit 1 **IACCVIOL**: Instruction access violation flag. This fault occurs on any access to an XN region, even the MPU is disabled or not present.
- When this bit is 1, the PC value stacked for the exception return points to the faulting instruction. The processor has not written a fault address to the MMAR.
- 0: No instruction access violation fault
 - 1: The processor attempted an instruction fetch from a location that does not permit execution.

4.4.14 Hard fault status register (HFSR)

Address offset: 0x2C

Reset value: 0x0000 0000

Required privilege: Privileged

The HFSR gives information about events that activate the hard fault handler. This register is read, write to clear. This means that bits in the register read normally, but writing 1 to any bit clears that bit to 0.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DEBU G_VT	FORC ED	Reserved													
rc_w1	rc_w1														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													VECT TBL	Res.	
													rc_w1		

Bit 31 DEBUG_VT: Reserved for Debug use. When writing to the register you must write 0 to this bit, otherwise behavior is unpredictable.

Bit 30 FORCED: Forced hard fault. Indicates a forced hard fault, generated by escalation of a fault with configurable priority that cannot be handles, either because of priority or because it is disabled.

When this bit is set to 1, the hard fault handler must read the other fault status registers to find the cause of the fault.

0: No forced hard fault

1: Forced hard fault.

Bits 29:2 Reserved, must be kept cleared

Bit 1 VECTTBL: Vector table hard fault. Indicates a bus fault on a vector table read during exception processing. This error is always handled by the hard fault handler.

When this bit is set to 1, the PC value stacked for the exception return points to the instruction that was preempted by the exception.

0: No bus fault on vector table read

1: Bus fault on vector table read.

Bit 0 Reserved, must be kept cleared

4.4.15 Memory management fault address register (MMFAR)

Address offset: 0x34

Reset value: undefined

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MMFAR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MMFAR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MMFAR**: Memory management fault address

When the MMARVALID bit of the MMFSR is set to 1, this field holds the address of the location that generated the memory management fault.

When an unaligned access faults, the address is the actual address that faulted. Because a single read or write instruction can be split into multiple aligned accesses, the fault address can be any address in the range of the requested access size.

Flags in the MMFSR register indicate the cause of the fault, and whether the value in the MMFAR is valid. See [Configurable fault status register \(CFSR; UFSR+BFSR+MMFSR\) on page 221](#).

4.4.16 Bus fault address register (BFAR)

Address offset: 0x38

Reset value: undefined

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BFAR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BFAR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **BFAR**: Bus fault address

When the BFARVALID bit of the BFSR is set to 1, this field holds the address of the location that generated the bus fault.

When an unaligned access faults the address in the BFAR is the one requested by the instruction, even if it is not the address of the fault.

Flags in the BFSR register indicate the cause of the fault, and whether the value in the BFAR is valid. See [Configurable fault status register \(CFSR; UFSR+BFSR+MMFSR\) on page 221](#).

4.4.17 Auxiliary fault status register (AFSR)

Address offset: 0x3C

Reset value: undefined

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IMPDEF[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IMPDEF[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **IMPDEF**: Implementation defined. The AFSR contains additional system fault information. The bits map to the AUXFAULT input signals.

This register is read, write to clear. This means that bits in the register read normally, but writing 1 to any bit clears that bit to 0.

Each AFSR bit maps directly to an AUXFAULT input of the processor, and a single-cycle HIGH signal on the input sets the corresponding AFSR bit to one. It remains set to 1 until you write 1 to the bit to clear it to zero.

When an AFSR bit is latched as one, an exception does not occur. Use an interrupt if an exception is required.

4.4.18 System control block design hints and tips

Ensure software uses aligned accesses of the correct size to access the system control block registers:

- except for the CFSR and SHPR1-SHPR3, it must use aligned word accesses
- for the CFSR and SHPR1-SHPR3 it can use byte or aligned halfword or word accesses.

The processor does not support unaligned accesses to system control block registers.

In a fault handler, to determine the true faulting address:

1. Read and save the MMFAR or BFAR value.
2. Read the MMARVALID bit in the MMFSR, or the BFARVALID bit in the BFSR. The MMFAR or BFAR address is valid only if this bit is 1.

Software must follow this sequence because another higher priority exception might change the MMFAR or BFAR value. For example, if a higher priority handler preempts the current fault handler, the other fault might change the MMFAR or BFAR value.

4.4.19 SCB register map

The table provides shows the System control block register map and reset values. The base address of the SCB register block is 0xE000 ED00 for register described in [Table 53](#).

Table 53. SCB register map and reset values

[illegible]

Table 53. SCB register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0x1C	SHPR2	PRI11								Reserved																										
	Reset Value	0	0	0	0	0	0	0	0																											
0x20	SHPR3	PRI15								PRI14								Reserved																		
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																			
0x24	SHCRS	Reserved														USG FAULT ENA	BUS FAULT ENA	MEM FAULT ENA	SV CALL PENDED	BUS FAULT PENDED	MEM FAULT PENDED	USG FAULT PENDED	SYS TICK ACT	PENDSV ACT	Reserved	MONITOR ACT	SV CALL ACT	Reserved				USG FAULT ACT	Reserved	BUS FAULT ACT	MEM FAULT ACT	
	Reset Value															0	0	0	0	0	0	0	0	0	0	0	0					0	0	0	0	0
0x28	CFSR	UFSR														BFSR								MMFSR												
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x2C	HFSR	DEBUG_VT	Reserved																																VECTBL	Reserved
	Reset Value	0																																	0	
0x34	MMAR	MMAR[31:0]																																		
	Reset Value	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x				
0x38	BFAR	BFAR[31:0]																																		
	Reset Value	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x				
0x3C	AFSR	IMPDEF[31:0]																																		
	Reset Value	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x				

4.5 SysTick timer (STK)

The processor has a 24-bit system timer, SysTick, that counts down from the reload value to zero, reloads (wraps to) the value in the STK_LOAD register on the next clock edge, then counts down on subsequent clocks.

When the processor is halted for debugging the counter does not decrement.

Table 54. System timer registers summary

Address	Name	Type	Required privilege	Reset value	Description
0xE000E010	STK_CTRL	RW	Privileged	0x00000000	<i>SysTick control and status register (STK_CTRL) on page 231</i>
0xE000E014	STK_LOAD	RW	Privileged	Unknown	<i>SysTick reload value register (STK_LOAD) on page 232</i>
0xE000E018	STK_VAL	RW	Privileged	Unknown	<i>SysTick current value register (STK_VAL) on page 233</i>
0xE000E01C	STK_CALIB	RO	Privileged	0xC0000000	<i>SysTick calibration value register (STK_CALIB) on page 234</i>

4.5.1 SysTick control and status register (STK_CTRL)

Address offset: 0x00

Reset value: 0x0000 0000

Required privilege: Privileged

The SysTick CTRL register enables the SysTick features.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															COUNT FLAG
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved												CLKSO URCE	TICK INT	EN ABLE	
												rw	rw	rw	

Bits 31:17 Reserved, must be kept cleared.

Bit 16 **COUNTFLAG**:

Returns 1 if timer counted to 0 since last time this was read.

Bits 15:3 Reserved, must be kept cleared.

Bit 2 **CLKSOURCE**: Clock source selection

Selects the clock source.

0: AHB/8

1: Processor clock (AHB)

Bit 1 **TICKINT**: SysTick exception request enable

0: Counting down to zero does not assert the SysTick exception request

1: Counting down to zero asserts the SysTick exception request.

Note: Software can use COUNTFLAG to determine if SysTick has ever counted to zero.

Bit 0 **ENABLE**: Counter enable

Enables the counter. When ENABLE is set to 1, the counter loads the RELOAD value from the LOAD register and then counts down. On reaching 0, it sets the COUNTFLAG to 1 and optionally asserts the SysTick depending on the value of TICKINT. It then loads the RELOAD value again, and begins counting.

0: Counter disabled

1: Counter enabled

4.5.2 SysTick reload value register (STK_LOAD)

Address offset: 0x04

Reset value: 0x0000 0000

Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								RELOAD[23:16]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RELOAD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept cleared.

Bits 23:0 **RELOAD**: RELOAD value

The LOAD register specifies the start value to load into the STK_VAL register when the counter is enabled and when it reaches 0.

Calculating the RELOAD value

The RELOAD value can be any value in the range 0x00000001-0x00FFFFFF. A start value of 0 is possible, but has no effect because the SysTick exception request and COUNTFLAG are activated when counting from 1 to 0.

The RELOAD value is calculated according to its use:

- I To generate a multi-shot timer with a period of N processor clock cycles, use a RELOAD value of N-1. For example, if the SysTick interrupt is required every 100 clock pulses, set RELOAD to 99.
- I To deliver a single SysTick interrupt after a delay of N processor clock cycles, use a RELOAD of value N. For example, if a SysTick interrupt is required after 100 clock pulses, set RELOAD to 99.

4.5.3 SysTick current value register (STK_VAL)

Address offset: 0x08
Reset value: 0x0000 0000
Required privilege: Privileged

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								CURRENT[23:16]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CURRENT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:24 Reserved, must be kept cleared.

Bits 23:0 **CURRENT**: Current counter value

The VAL register contains the current value of the SysTick counter.

Reads return the current value of the SysTick counter.

A write of any value clears the field to 0, and also clears the COUNTFLAG bit in the STK_CTRL register to 0.

4.5.4 SysTick calibration value register (STK_CALIB)

Address offset: 0x0C

Reset value: 0x0000000

Required privilege: Privileged

The CALIB register indicates the SysTick calibration properties.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
NO REF	SKEW	Reserved						TENMS[23:16]							
r	r							r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TENMS[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **NOREF**: NOREF flag. Reads as zero. Indicates that a separate reference clock is provided. The frequency of this clock is HCLK/8.

Bit 30 **SKEW**: SKEW flag: Indicates whether the TENMS value is exact. Reads as one. Calibration value for the 1 ms inexact timing is not known because TENMS is not known. This can affect the suitability of SysTick as a software real time clock.

Bits 29:24 Reserved, must be kept cleared.

Bits 23:0 **TENMS[23:0]**: Calibration value. Indicates the calibration value when the SysTick counter runs on HCLK max/8 as external clock. The value is product dependent, please refer to the Product Reference Manual, SysTick Calibration Value section. When HCLK is programmed at the maximum frequency, the SysTick period is 1ms.
If calibration information is not known, calculate the calibration value required from the frequency of the processor clock or external clock.

4.5.5 SysTick design hints and tips

The SysTick counter runs on the processor clock. If this clock signal is stopped for low power mode, the SysTick counter stops.

Ensure software uses aligned word accesses to access the SysTick registers.

The SysTick counter reload and current value are undefined at reset, the correct initialization sequence for the SysTick counter is:

1. Program reload value.
2. Clear current value.
3. Program Control and Status register.

4.5.6 SysTick register map

The table provided shows the SysTick register map and reset values. The base address of the SysTick register block is 0xE000 E010.

Table 55. SysTick register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	STK_CTRL Reset Value	Reserved															COUNTFLAG 0	Reserved													CLKSOURCE 1	TICKINT 0	ENABLE 0
0x04	STK_LOAD Reset Value	Reserved									RELOAD[23:0]																						
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x08	STK_VAL Reset Value	Reserved									CURRENT[23:0]																						
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C	STK_CALIB Reset Value	Reserved									TENMS[23:0]																						
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

4.6 Floating point unit (FPU)

The Cortex-M4F FPU implements the FPUv4-SP floating-point extension.

The FPU fully supports single-precision add, subtract, multiply, divide, multiply and accumulate, and square root operations. It also provides conversions between fixed xxxxx-point and floating-point data formats, and floating-point constant instructions.

The FPU provides floating-point computation functionality that is compliant with the ANSI/IEEE standard 754-2008, IEEE standard for Binary Floating-Point Arithmetic, referred to as the IEEE 754 standard.

The FPU contains 32 single-precision extension registers, which you can also access as 16 doubleword registers for load, store, and move operations.

[Table 56](#) shows the floating-point system registers in the Cortex-M4F system control block (SCB). The base address of the additional registers for the FP extension is 0xE000 ED00.

Table 56. Cortex-M4F floating-point system registers

Address	Name	Type	Reset	Description
0xE000ED88	CPACR	RW	0x00000000	Section 4.6.1: Coprocessor access control register (CPACR) on page 237
0xE000EF34	FPCCR	RW	0xC0000000	Section 4.6.2: Floating-point context control register (FPCCR) on page 237
0xE000EF38	FPCAR	RW	-	Section 4.6.3: Floating-point context address register (FPCAR) on page 239
0xE000EF3C	FPDSCR	RW	0x00000000	Section 4.6.5: Floating-point default status control register (FPDSCR) on page 241
-	FPSCR	RW	-	Section 4.6.4: Floating-point status control register (FPSCR) on page 239

The following sections describe the floating-point system registers whose implementation is specific to this processor.

Note: For more details on the IEEE standard and floating-point arithmetic (IEEE 754), refer to the AN4044 Application note. Available from website www.st.com.

4.6.1 Coprocessor access control register (CPACR)

Address offset (from SCB): 0x88

Reset value: 0x0000000

Required privilege: Privileged

The CPACR register specifies the access privileges for coprocessors.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								CP11		CP10		Reserved			
								rw		rw					
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

Bits 31:24 Reserved. Read as Zero, Write Ignore.

Bits 23:20 **CPn**: [2n+1:2n] for n values 10 and 11. Access privileges for coprocessor n. The possible values of each field are:

0b00: Access denied. Any attempted access generates a NOCP UsageFault.

0b01: Privileged access only. An unprivileged access generates a NOCP fault.

0b10: Reserved. The result of any access is Unpredictable.

0b11: Full access.

Bits 19:0 Reserved. Read as Zero, Write Ignore.

4.6.2 Floating-point context control register (FPCCR)

Address offset: 0x04

Reset value: 0xC000000

Required privilege: Privileged

The FPCCR register sets or returns FPU control data.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ASPEN	LSPEN	Reserved													
rw	rw														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								MONRDY	Reserved	BFRDY	MMRDY	HFRDY	THREAD	Reserved	USER
								rw		rw	rw	rw	rw		rw
										rw					LSPACT
															rw

Bit 31 **ASPEN**: Enables CONTROL<2> setting on execution of a floating-point instruction. This results in automatic hardware state preservation and restoration, for floating-point context, on exception entry and exit.

0: Disable CONTROL<2> setting on execution of a floating-point instruction.

1: Enable CONTROL<2> setting on execution of a floating-point instruction.

Bit 30 **LSPEN**:

0: Disable automatic lazy state preservation for floating-point context.

1: Enable automatic lazy state preservation for floating-point context.

Bits 29:9 Reserved.

Bit 8 **MONRDY**:

0: DebugMonitor is disabled or priority did not permit setting MON_PEND when the floating-point stack frame was allocated.

1: DebugMonitor is enabled and priority permits setting MON_PEND when the floating-point stack frame was allocated.

Bit 7 Reserved.

Bit 6 **BFRDY**:

0: BusFault is disabled or priority did not permit setting the BusFault handler to the pending state when the floating-point stack frame was allocated.

1: BusFault is enabled and priority permitted setting the BusFault handler to the pending state when the floating-point stack frame was allocated.

Bit 5 **MMRDY**:

0: MemManage is disabled or priority did not permit setting the MemManage handler to the pending state when the floating-point stack frame was allocated.

1: MemManage is enabled and priority permitted setting the MemManage handler to the pending state when the floating-point stack frame was allocated.

Bit 4 **HFRDY**:

0: Priority did not permit setting the HardFault handler to the pending state when the floating-point stack frame was allocated.

1: Priority permitted setting the HardFault handler to the pending state when the floating-point stack frame was allocated.

Bit 3 **THREAD**:

0: Mode was not Thread Mode when the floating-point stack frame was allocated.

1: Mode was Thread Mode when the floating-point stack frame was allocated.

Bit 2 Reserved.

Bit 1 **USER**:

0: Privilege level was not user when the floating-point stack frame was allocated.

1: Privilege level was user when the floating-point stack frame was allocated.

Bit 1 **LSPACT**:

0: Lazy state preservation is not active.

1: Lazy state preservation is active. floating-point stack frame has been allocated but saving state to it has been deferred.

4.6.3 Floating-point context address register (FPCAR)

Address offset: 0x08

Reset value: 0x00000000

Required privilege: Privileged

The FPCAR register holds the location of the unpopulated floating-point register space allocated on an exception stack frame.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADDRESS[31:16]															
rw															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDRESS[15:3]													Reserved		
rw															

Bits 31:3 ADDRESS: Location of unpopulated floating-point register space allocated on an exception stack frame.

Bits 2:0 Reserved. Read as Zero, Writes Ignored.

4.6.4 Floating-point status control register (FPSCR)

Address offset: Not mapped

Reset value: 0x00000000

Required privilege: Privileged

The FPSCR register provides all necessary user level control of the floating-point system.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
N	Z	C	V	Reserved	AHP	DN	FZ	RMode		Reserved					
rw	rw	rw	rw		rw	rw	rw	rw	rw						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								IDC	Reserved		IXC	UFC	OFC	DZC	IOC
								rw			rw	rw	rw	rw	rw

Bit 31 **N**: Negative condition code flag. Floating-point comparison operations update these flags. For more details on the result, refer to [Table 57](#).

0: Operation result was positive, zero, greater than, or equal.

1: Operation result was negative or less than.

Bit 30 **Z**: Zero condition code flag. Floating-point comparison operations update these flags. For more details on the result, refer to [Table 57](#).

0: Operation result was not zero.

1: Operation result was zero.

Bit 29 **C**: Carry condition code flag. Floating-point comparison operations update these flags. For more details on the result, refer to [Table 57](#).

0: Add operation did not result in a carry bit or subtract operation resulted in a borrow bit.

1: Add operation resulted in a carry bit or subtract operation did not result in a borrow bit.

- Bit 28 **V**: Overflow condition code flag. Floating-point comparison operations update this flag. For more details on the result, refer to [Table 57](#).
 0: Operation did not result in an overflow
 1: Operation resulted in an overflow.
- Bit 27 Reserved.
- Bit 26 **AHP**: Alternative half-precision control bit:
 0: IEEE half-precision format selected.
 1: Alternative half-precision format selected.
- Bit 25 **DN**: Default NaN mode control bit:
 0: NaN operands propagate through to the output of a floating-point operation.
 1: Any operation involving one or more NaNs returns the Default NaN.
- Bit 24 **FZ**: Flush-to-zero mode control bit:
 0: Flush-to-zero mode disabled. Behavior of the floating-point system is fully compliant with the IEEE 754 standard.
 1: Flush-to-zero mode enabled.
- Bits 23:22 **RMode**: Rounding Mode control field. The specified rounding mode is used by almost all floating-point instructions:
 0b00: Round to nearest (RN) mode
 0b01: Round towards plus infinity (RP) mode
 0b10: Round towards minus infinity (RM) mode
 0b11: Round towards zero (RZ) mode.
- Bit 21:8 Reserved.
- Bit 7 **IDC**: Input denormal cumulative exception bit. Cumulative exception bit for floating-point exception.
 1: Indicates that the corresponding exception occurred since 0 was last written to it.
- Bit 6:5 Reserved
- Bit 4 **IXC**: Inexact cumulative exception bit. Cumulative exception bit for floating-point exception.
 1: Indicates that the corresponding exception occurred since 0 was last written to it.
- Bit 3 **UFC**: Underflow cumulative exception bit. Cumulative exception bit for floating-point exception.
 1: Indicates that the corresponding exception occurred since 0 was last written to it.
- Bit 2 **OFC**: Overflow cumulative exception bit. Cumulative exception bit for floating-point exception.
 1: Indicates that the corresponding exception occurred since 0 was last written to it.
- Bit 1 **DZC**: Division by zero cumulative exception bit. Cumulative exception bit for floating-point exception. 1: Indicates that the corresponding exception occurred since 0 was last written to it.
- Bit 0 **IOC**: Invalid operation cumulative exception bit. Cumulative exception bit for floating-point exception. 1: Indicates that the corresponding exception occurred since 0 was last written to it.

Table 57. Effect of a Floating-point comparison on the condition flags

Comparison result	N	Z	C	V
Equal	0	1	1	0
Less than	1	0	0	0
Greater than	0	0	1	0
Unordered	0	0	1	1

4.6.5 Floating-point default status control register (FPDSCR)

Address offset: 0x0C

Reset value: 0x0000000

Required privilege: Privileged

The FPDSCR register holds the default values for the floating-point status control data.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					AHP	DN	FZ	RMode		Reserved					
					rw	rw	rw	rw	rw						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

Bits 31:27 Reserved, must be kept cleared.

Bit 26 **AHP**: Default value for FPSCR.AHP

Bit 25 **DN**: Default value for FPSCR.DN

Bit 24 **FZ**: Default value for FPSCR.FZ

Bits 23:22 **RMode**: Default value for FPSCR.RMode

Bits 21:0 Reserved, must be kept cleared.

4.6.6 Enabling the FPU

The FPU is disabled from reset. You must enable it before you can use any floating-point instructions.

The example shows an example code sequence for enabling the FPU in both privileged and user modes. The processor must be in privileged mode to read from and write to the CPACR.

Example

```

; CPACR is located at address 0xE000ED88
LDR.W  R0, =0xE000ED88
; Read CPACR
LDR    R1, [R0]
; Set bits 20-23 to enable CP10 and CP11 coprocessors
ORR    R1, R1, #(0xF << 20)
; Write back the modified value to the CPACR
STR    R1, [R0]; wait for store to complete
DSB
;reset pipeline now the FPU is enabled
ISB

```

4.6.7 Enabling and clearing FPU exception interrupts

The FPU exception flags (IDC, UFC, OFC, DZC and additionally, in STM32F3xxx devices, the IXC flag) are ORed and connected to the interrupt controller.

In STM32F4xx devices there is no individual mask and the enable/disable of the FPU interrupt is done at interrupt controller level.

In STM32F3xx devices in addition to the FPU interrupt controller interrupt mask bit, there is an individual mask bit for enabling/disabling the FPU interrupt sources (the IXC flag is masked by default in STM32F3xx devices).

As it occurs very frequently, the IXC exception flag is not connected to the interrupt controller in STM32F4xx devices, and cannot generate an interrupt. If needed, it must be managed by polling.

Clearing the FPU exception flags depends on the FPU context save/restore configuration:

- No floating-point register saving: when Floating-point context control register (FPCCR) Bit 30 LSPEN=0 and Bit 31 ASPEN=0.

You must clear interrupt source in Floating-point Status and Control Register (FPSCR).

Example:

```
register uint32_t fpscr_val = 0;
fpscr_val = __get_FPSCR();
{ check exception flags }
fpscr_val &= (uint32_t)~0x8F; // Clear all exception flags
__set_FPSCR(fpscr_val);
```

- Lazy save/restore: when Floating-point context control register (FPCCR) Bit 30 LSPEN=1 and Bit 31 ASPEN=X.

In the case of lazy floating-point context save/restore, a dummy read access should be made to Floating-point Status and Control Register (FPSCR) to force state preservation and FPSCR clear.

Then handle FPSCR in the stack.

Example:

```
register uint32_t fpscr_val = 0;
register uint32_t reg_val = 0;
reg_val = __get_FPSCR(); //dummy access
fpscr_val=*(__IO uint32_t*)(FPU->FPCAR +0x40);
{ check exception flags }
fpscr_val &= (uint32_t)~0x8F ; // Clear all exception flags
*(__IO uint32_t*)(FPU->FPCAR +0x40)=fpscr_val;
__DMB() ;
```

- Automatic floating-point registers save/restore: when Floating-point context control register (FPCCR)

Bit 30 LSPEN=0 and Bit 31 ASPEN=1.

In case of automatic floating-point context save/restore, a read access should be made to Floating-point Status and Control Register (FPSCR) to force clear.

Then handle FPSCR in the stack.

Example:

```
// FPU Exception handler
void FPU_ExceptionHandler(uint32_t lr, uint32_t sp)
{
register uint32_t fpscr_val;
if(lr == 0xFFFFF9E9)
{
sp = sp + 0x60;
```

```
    }
    else if(lr == 0xFFFFFFFF)
    {
        sp = __get_PSP() + 0x60 ;
    }
    fpscr_val = *(uint32_t*)sp;
    { check exception flags }
    fpscr_val &= (uint32_t)~0x8F ;    // Clear all exception flags
    *(uint32_t*)sp = fpscr_val;
    __DMB() ;
}
// FPU IRQ Handler
void __asm FPU_IRQHandler(void)
{
    IMPORT FPU_ExceptionHandler
    MOV    R0, LR                // move LR to R0
    MOV    R1, SP                // Save SP to R1 to avoid any modification to
                                // the stack pointer from FPU_ExceptionHandler
    VMRS   R2, FPSCR             // dummy read access, to force clear
    B      FPU_ExceptionHandler
    BX     LR
}
```

5 Revision history

Table 58. Document revision history

Date	Revision	Changes
20-Feb-2012	1	Initial release.
09-Jul-2012	2	Changed reset value in Section 4.6.2: Floating-point context control register (FPCCR) . Added Table 1: Applicable products .
04-Sep-2012	3	Added information on the STM32F3xxx Cortex-M4 processor. Added extra part numbers to Table 1: Applicable products . Added related documentation references to Introduction . Changed "IEEE754-compliant single-precision FPU" bullet in Section 1.3.3: Cortex-M4 processor features and benefits summary . Added information on extended interrupt/event controller to Section 2.5.3: External event input / extended interrupt and event input . Changed first "interrupt" bullet in Section 4.3: Nested vectored interrupt controller (NVIC) . Removed outdated reset value information in Section 4.4.7: Configuration and control register (CCR) , and for 0x14 offset in Table 52: System fault handler priority fields . Added a note about IEEE 754 to Section 4.6: Floating point unit (FPU) .
12-May-2014	4	Updated Reference documents . Updated Section 4.4.1: Auxiliary control register (ACTLR) . Updated Section 4.5.1: SysTick control and status register (STK_CTRL) .

Please Read Carefully:

Information in this document is provided solely in connection with ST products. STMicroelectronics NV and its subsidiaries ("ST") reserve the right to make changes, corrections, modifications or improvements, to this document, and the products and services described herein at any time, without notice.

All ST products are sold pursuant to ST's terms and conditions of sale.

Purchasers are solely responsible for the choice, selection and use of the ST products and services described herein, and ST assumes no liability whatsoever relating to the choice, selection or use of the ST products and services described herein.

No license, express or implied, by estoppel or otherwise, to any intellectual property rights is granted under this document. If any part of this document refers to any third party products or services it shall not be deemed a license grant by ST for the use of such third party products or services, or any intellectual property contained therein or considered as a warranty covering the use in any manner whatsoever of such third party products or services or any intellectual property contained therein.

UNLESS OTHERWISE SET FORTH IN ST'S TERMS AND CONDITIONS OF SALE ST DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY WITH RESPECT TO THE USE AND/OR SALE OF ST PRODUCTS INCLUDING WITHOUT LIMITATION IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE (AND THEIR EQUIVALENTS UNDER THE LAWS OF ANY JURISDICTION), OR INFRINGEMENT OF ANY PATENT, COPYRIGHT OR OTHER INTELLECTUAL PROPERTY RIGHT.

ST PRODUCTS ARE NOT DESIGNED OR AUTHORIZED FOR USE IN: (A) SAFETY CRITICAL APPLICATIONS SUCH AS LIFE SUPPORTING, ACTIVE IMPLANTED DEVICES OR SYSTEMS WITH PRODUCT FUNCTIONAL SAFETY REQUIREMENTS; (B) AERONAUTIC APPLICATIONS; (C) AUTOMOTIVE APPLICATIONS OR ENVIRONMENTS, AND/OR (D) AEROSPACE APPLICATIONS OR ENVIRONMENTS. WHERE ST PRODUCTS ARE NOT DESIGNED FOR SUCH USE, THE PURCHASER SHALL USE PRODUCTS AT PURCHASER'S SOLE RISK, EVEN IF ST HAS BEEN INFORMED IN WRITING OF SUCH USAGE, UNLESS A PRODUCT IS EXPRESSLY DESIGNATED BY ST AS BEING INTENDED FOR "AUTOMOTIVE, AUTOMOTIVE SAFETY OR MEDICAL" INDUSTRY DOMAINS ACCORDING TO ST PRODUCT DESIGN SPECIFICATIONS. PRODUCTS FORMALLY ESCC, QML OR JAN QUALIFIED ARE DEEMED SUITABLE FOR USE IN AEROSPACE BY THE CORRESPONDING GOVERNMENTAL AGENCY.

Resale of ST products with provisions different from the statements and/or technical features set forth in this document shall immediately void any warranty granted by ST for the ST product or service described herein and shall not create or extend in any manner whatsoever, any liability of ST.

ST and the ST logo are trademarks or registered trademarks of ST in various countries.

Information in this document supersedes and replaces all information previously supplied.

The ST logo is a registered trademark of STMicroelectronics. All other names are the property of their respective owners.

© 2014 STMicroelectronics - All rights reserved

STMicroelectronics group of companies

Australia - Belgium - Brazil - Canada - China - Czech Republic - Finland - France - Germany - Hong Kong - India - Israel - Italy - Japan - Malaysia - Malta - Morocco - Philippines - Singapore - Spain - Sweden - Switzerland - United Kingdom - United States of America

www.st.com

